RELOOP: Recursive Retrieval with Multi-Hop Reasoner and Planners for Heterogeneous QA

Ruiyi Yang[1] **Hao Xue**[2,1] **Imran Razzak**[3,1] **Hakim Hacid**[4] **Flora D. Salim**[1]
[1]UNSW Sydney, Sydney, NSW, Australia
[2]The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China
[3]Mohamed Bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE
[4]Technology Innovation Institute, Abu Dhabi, UAE
{ruiyi.yang, flora.salim}@unsw.edu.au
haoxue@hkust-gz.edu.cn
imran.razzak@mbzuai.ac.ae; hakim.hacid@tii.ae

Abstract

Retrieval-augmented generation (RAG) remains challenging for multi-hop question answering when supporting evidence is distributed across heterogeneous sources, including text, tables, and knowledge graphs (KGs). Existing systems often rely on modality-specific retrieval pipelines or uncontrolled iterative search, making it difficult to preserve source structure while maintaining efficient evidence acquisition. We present **RELOOP**, a structure-aware retrieval framework built on **Hierarchical Sequence (HSEQ)**, a reversible representation that maps heterogeneous sources into provenance-preserving typed segments with lightweight structural links. Operating on this unified evidence interface, a lightweight iterator selects and expands candidate evidence through structure-respecting actions, while an explicit sufficiency signal determines when the accumulated evidence is adequate for answering. A head module then synthesizes the final answer from a compact, canonicalized evidence package, with an optional refinement step when inconsistent evidence is detected. Experiments on HotpotQA, HybridQA, TAT-QA, and MetaQA show that RELOOP consistently improves answer quality over strong single-pass and iterative RAG baselines while requiring substantially fewer retrieval steps than graph-expansion approaches. These results suggest that a reversible, structure-aware evidence interface provides an effective foundation for accurate and budget-controlled retrieval across heterogeneous QA settings. Code is available at https://github.com/cruiseresearchgroup/RELOOP.

1 Introduction

Retrieval-augmented generation (RAG) improves the factual grounding of large language models (LLMs) by conditioning answer generation on external evidence (Lewis et al., 2021; Jiang et al., 2023; Gao et al., 2023). However, many question answering tasks require multi-step reasoning over heterogeneous sources, including unstructured text, relational tables, and knowledge graphs (KGs). In these settings, relevant evidence is often distributed across multiple units or formats, requiring a system to retrieve related information, preserve structural context, and determine when sufficient evidence has been collected. Existing RAG pipelines remain limited in three aspects. First, single-pass retrieve-then-generate systems (Luo et al., 2024a; Glass et al., 2022) retrieve a fixed set of chunks and often fail to cover complete evidence chains for multi-hop questions. Second, iterative or agentic retrieval systems (Liu et al., 2025; Yang et al., 2025; Chen et al., 2025; Singh et al., 2025) can recover additional evidence, but repeated planning and expansion may introduce redundant retrieval steps and unpredictable inference costs. Third, heterogeneous evidence is commonly handled using separate representations and retrieval procedures for text, tables, and KGs (Yu, 2022; Christmann and Weikum, 2024), limiting policy reuse across formats and weakening provenance when structured sources are flattened into plain text.

A central challenge is therefore to construct an evidence interface that is directly consumable by LLMs while retaining the structural links needed for controlled navigation. Flat serialization preserves sequential readability but discards useful locality, such as paragraph–sentence relations, table coordinates, and KG neighborhoods. Graph-based representations expose connectivity, but may obscure the sequential organization of text and tables. An effective interface for heterogeneous QA should preserve both sequence compatibility and modality-specific structure.

RELOOP addresses this challenge through **Hierarchical Sequence (HSEQ)**, a reversible representation that maps text, tables, and KGs into a common sequence of typed, provenance-preserving segments. Each segment retains lightweight struc-

Findings of the Association for Computational Linguistics: EMNLP 2026, pages 37120–37138
October 24-29, 2026 ©2026 Association for Computational Linguistics

tural metadata, such as parent–child relations, row/column coordinates, or relational neighborhoods. On this interface, a budget-aware iterator selects and expands evidence through structure-respecting actions, while an explicit sufficiency signal controls termination. A compact guidance plan provides a soft prior over promising retrieval directions, and the selected evidence is canonicalized into a provenance-preserving package for final answer synthesis. In our instantiation, RELOOP-I is not a zero-shot controller: it is LoRA-adapted with supervised action and sufficiency trajectories derived from training data, while validation and test annotations are excluded from trajectory construction. Experiments on HotpotQA, HybridQA, TAT-QA, and MetaQA evaluate RELOOP across text, table–text, and KG question answering. The results show consistent answer-quality improvements over strong single-pass and iterative retrieval baselines, together with substantially fewer retrieval steps than graph-expansion approaches. The main contributions are as follows:

- **Reversible unified evidence interface.** HSEQ represents text, tables, and KGs as typed sequential segments while preserving modality-specific structural links and source provenance.
- **Sufficiency-controlled iterative retrieval.** RELOOP performs budget-aware evidence selection and structural expansion over HSEQ, using guidance as a soft prior and an explicit sufficiency signal for termination.
- **Evaluation across heterogeneous QA settings.** RELOOP is evaluated on text, table–text, and KG benchmarks, with task-wise accuracy/F1 results, stopping diagnostics, and retrieval-cost analyses.

2 Related Work

Retrieval over heterogeneous and structured evidence. Retrieval-augmented generation (RAG) grounds language-model outputs in external evidence and has been extended from unstructured passages to heterogeneous knowledge sources (Lewis et al., 2021; Jiang et al., 2023; Yu, 2022). For table–text reasoning, systems such as TAT-LLM and TableRAG target numerical or cross-format evidence aggregation (Zhu et al., 2024; Yu et al., 2025). Graph-based RAG methods organize retrieved information through entities, relations, communities, or chunk-level connectivity, enabling relational expansion and global evidence aggregation (Edge et al., 2024; Hu et al., 2024; Mavromatis and Karypis, 2024; Luo et al., 2025). These approaches demonstrate the value of structured retrieval, but commonly rely on modality-specific representations or graph-centric transformations rather than a shared sequence-compatible interface for text, tables, and KGs.

Iterative and plan-guided retrieval. Multi-hop QA often requires evidence to be acquired through multiple retrieval steps instead of a single retrieve-then-generate pass. Adaptive-RAG selects retrieval strategies according to question complexity (Jeong et al., 2024), while Think-on-Graph performs iterative reasoning through KG-guided traversal (Ma et al., 2024). Closely related to RELOOP, TRACE constructs knowledge-grounded reasoning chains from retrieved documents through an intermediate graph representation (Fang et al., 2024), and REAP recursively alternates sub-task planning with fact extraction to support multi-hop textual QA (Zhu et al., 2026). These methods improve multi-step evidence acquisition, but their retrieval interfaces are mainly designed for text or graph-specific settings. They do not target a reversible cross-format evidence interface on which the same structure-respecting retrieval operations can be expressed for text, tables, and KGs.

Structured retrieval interfaces. Hierarchical and structural RAG systems introduce additional organization over retrieved contexts, such as document hierarchies, modular indices, or chunk-level graphs (Reynolds and Corrigan, 2024; Chen et al., 2024; Liu et al., 2025; Huang et al., 2025). Such structures improve retrieval granularity and context organization, but are often used as indexing or routing mechanisms before final evidence is converted into opaque textual contexts for the LLM. In contrast, HSEQ keeps the retrieved evidence itself in a typed sequential form while retaining structural links and provenance. This design makes the evidence directly readable by the language model and, at the same time, exposes navigation operations such as parent–child expansion, row/column traversal, and relation-hop expansion.

Positioning of RELOOP. RELOOP differs from prior structured and iterative RAG methods by jointly targeting heterogeneous evidence representation and controlled multi-step retrieval. Specifically, it provides a reversible, provenance-

Method	Het.	Struct.	Iter.	Rev.	Suff.
GraphRAG (Edge et al., 2024)	×	○	×	×	×
Adaptive-RAG (Jeong et al., 2024)	×	×	✓	×	×
Think-on-Graph (Ma et al., 2024)	×	✓	✓	×	×
TRACE (Fang et al., 2024)	×	○	✓	×	×
REAP (Zhu et al., 2026)	×	○	✓	×	○
RELOOP	✓	✓	✓	✓	✓

Table 1: Design-level comparison with representative structured and iterative RAG methods. Het.: unified support for text, tables, and KGs; Struct.: explicit structure-aware evidence navigation; Iter.: iterative evidence acquisition; Rev.: reversible provenance-preserving representation; Suff.: explicit sufficiency-controlled stopping. ✓ indicates explicit support, ○ partial or indirect support, and × no corresponding design component.

preserving interface for text, tables, and KGs, together with structure-aware iteration governed by an explicit sufficiency signal. Table 1 summarizes these design-level distinctions.

3 RELOOP Framework

3.1 Task Formulation and Framework Overview

Heterogeneous QA under resource budgets. Given a natural-language query q and a heterogeneous corpus $D = \{(x_j, m_j)\}_{j=1}^{N}$, where $m_j \in \{\texttt{text}, \texttt{table}, \texttt{kg}\}$ denotes the modality of source item x_j, the task is to produce an answer $y \in \mathcal{Y}$ supported by evidence selected from D. Inference is subject to a resource budget B, which may constrain the number of retrieval steps, selected segments, tokens, tool calls, or online latency. Let $E^\star \subseteq D$ denote a minimally sufficient evidence set for answering q under a fixed answerer.

Framework overview. RELOOP formulates evidence acquisition as a short sequence of structure-aware selection and expansion steps. A modality-aware adapter τ first maps the heterogeneous corpus into a hierarchical sequence $S_h = \tau(D)$. On this interface, an iterator π_θ accumulates a compact evidence set $M^\star$ under budget B, guided by a retrieval prior g and an explicit sufficiency criterion Φ. A canonicalizer κ subsequently transforms the selected segments into a compact, provenance-preserving evidence package for a head module $\mathcal{H}$, which produces the final answer. Formally, the framework is defined as

$$\mathcal{F} = \big(\tau, \pi_\theta, \Phi, \kappa, \mathcal{H}\big), \qquad \mathcal{F}(q, D) = \mathcal{H}(q, \kappa(M^\star)), \tag{1}$$

Src.	Level	Content $c(s)$	Structure / provenance in $\mu(s)$
Text	sent.	sentence span	parent paragraph; offsets
Table	cell	cell value	row/column ids; linked passage
KG	triple	(h, r, t)	entity/relation ids; adjacent triples

Table 2: Schematic HSEQ segments across sources.

where

$$S_h = \tau(D), \qquad M^\star = \pi_\theta(q, S_h;\, g, \Phi, B)\,. \tag{2}$$

During inference, π_θ operates over candidate segments in S_h and may expand selected evidence through modality-specific structural links, including paragraph–sentence relations, table row/column coordinates, and KG neighborhoods. Iteration terminates when Φ judges the accumulated evidence to be sufficient or when the available budget is exhausted. Although the structural links differ across source formats, HSEQ exposes them through a common action schema consisting of evidence selection, structural expansion, and sufficiency-controlled termination. RELOOP consists of three modules: **HSEQ-Adapter (HSEQ-A)**, which constructs the unified evidence interface; **RELOOP-Iterator (RELOOP-I)**, which performs budget-aware evidence acquisition; and **RELOOP-Head (RELOOP-H)**, which generates guidance in the planner-based setting and produces the final evidence-conditioned answer.

3.2 HSEQ-Adapter (HSEQ-A)

HSEQ-Adapter (HSEQ-A) converts heterogeneous sources into a unified hierarchical sequence S_h that exposes structural locality and source provenance in a modality-compatible format. Formally, each source item x_j is mapped by a modality-specific adapter τ_{m_j} to a finite set of segments $\tau_{m_j}(x_j) \subset \mathcal{S}$, which are concatenated as $S_h = \bigsqcup_{j=1}^{N} \tau_{m_j}(x_j) \in \mathcal{S}^*$. Each segment $s \in \mathcal{S}$ is represented as $s = \big(\mathrm{id}(s), \ell(s), p(s), c(s), \mu(s)\big)$, where $\ell(s)$ is its structural level, $p(s)$ is an optional parent pointer, $c(s)$ is human-readable content, and $\mu(s)$ records provenance and alignment metadata, such as text offsets, table coordinates, or KG entity/relation identifiers.

HSEQ is *reversible* in the sense that each selected segment can be traced back to its original evidence unit through its identifiers and alignment metadata. Segments are arranged in parent-before-child order, and their retained structure defines neighborhood operations for subsequent retrieval, including paragraph–sentence expansion,

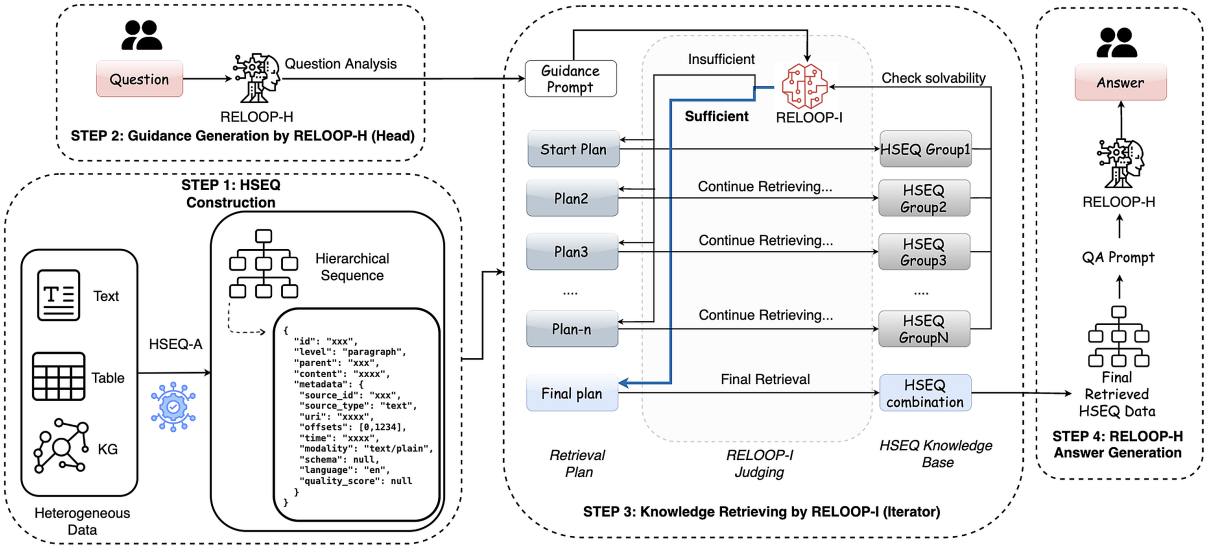


Figure 1: Overview of RELOOP. (*i*) HSEQ-A maps heterogeneous sources into a hierarchical sequence S_h of typed segments with source provenance and structural links; (*ii*) RELOOP-I iteratively selects and expands candidate evidence under resource budgets, using guidance g as a soft prior and Φ to estimate evidence sufficiency; (*iii*) the canonicalizer κ converts the selected segments into a compact evidence package; (*iv*) RELOOP-H synthesizes the answer and may trigger a brief refinement step when inconsistent evidence is detected.

table row/column navigation, and adjacent-relation expansion in KGs. More detailed HSEQ examples are provided in App. A.9.

3.3 RELOOP-Iterator (RELOOP-I)

RELOOP-Iterator (RELOOP-I) performs budget-aware evidence acquisition over S_h through guidance-seeded candidate selection, structural expansion, and sufficiency-controlled stopping.

Guidance prior. Each retrieval episode is conditioned on a short guidance plan $g = g(q, \text{type})$, which specifies initial targets, useful expansion directions, and candidate stopping conditions. The guidance may be produced by the head module $\mathcal{H}$, retrieved from previously generated guidance, or instantiated from heuristic templates keyed by question type. Although g guides candidate selection and expansion, it is not itself supporting evidence: termination additionally depends on whether the retrieved segments satisfy the sufficiency prediction. Consequently, RELOOP-I can deviate from inaccurate guidance when the observed evidence does not support the suggested direction.

Iteration control. Let $M_t \subseteq S_h$ denote the selected evidence at step t, $C_t \subseteq S_h$ a candidate window under the current budget state B_t, and $\mathcal{N}(\cdot)$ the structure-aware neighborhood operators induced by HSEQ. Rather than scanning S_h from its beginning, the initial window C_0 is seeded by query- and guidance-relevant segments. At each step, RELOOP-I follows

$$\pi_\theta(a_t, s_t \mid q, S_h, M_t, C_t, g, B_t), \tag{3}$$

where a_t selects up to k segments from C_t and may expand them through $\mathcal{N}(\cdot)$, while $s_t \in \{0, 1\}$ predicts whether the accumulated evidence is sufficient for answering. The selected evidence and candidate window are updated as

$$M_{t+1} = u(M_t, a_t), \qquad C_{t+1} = \text{Refresh}(S_h, M_{t+1}, B_{t+1}, \rho) \tag{4}$$

where ρ determines the local presentation order of segments within the seeded and expanded regions. Iteration terminates when $s_t = 1$ after the minimum-step constraint is satisfied, or when the available budget is exhausted. With window size W and step cap $T_{\max}$, Alg. 1 summarizes the iterative selection procedure. The bounded candidate window limits per-step context size, while structural expansion allows RELOOP-I to move directly among related text spans, table regions, or KG neighborhoods.

Illustrative expansion. Given a seeded candidate segment, RELOOP-I may expand a sentence to its parent paragraph, a table cell to its associated row or column, or a KG triple to adjacent relations. The resulting segments are added to M_t, after which the sufficiency prediction determines whether further retrieval is required.

Algorithm 1 Guided Iterative Selection under RELOOP-I

Require: question q, HSEQ S_h, guidance g, initial budget B_0, window size W, step cap $T_{\max}$, minimum steps $T_{\min}$, top-k k, ordering ρ

$M_0 \leftarrow \varnothing$
$C_0 \leftarrow \mathrm{SeedWindow}(q, S_h, g; W, B_0, \rho)$
for $t = 1$ to $T_{\max}$ **do**
  $(K_t, s_t) \leftarrow \pi_\theta(q, S_h, M_{t-1}, C_{t-1}, g, B_{t-1})$ ▷ $K_t \subseteq C_{t-1},\ |K_t| \le k$
  $M_t \leftarrow M_{t-1} \cup K_t$
  if $s_t = 1$ **and** $t \ge T_{\min}$ **then**
    return $\kappa(M_t)$
  end if
  $B_t \leftarrow \mathrm{UpdateBudget}(B_{t-1}, K_t)$
  $C_t \leftarrow \mathrm{Refresh}(S_h, M_t; W, B_t, \rho)$
end for
return $\kappa(M_{T_{\max}})$

3.4 RELOOP-Head (RELOOP-H)

RELOOP-Head (RELOOP-H) constructs guidance in the planner-based setting and performs evidence-conditioned answer synthesis.

Guidance construction. Each question is paired with a compact guidance plan g describing initial retrieval targets, useful evidence types, expansion directions, and candidate stopping conditions. In the planner-based setting, $\mathcal{H}$ generates g from the input question. For latency-sensitive inference, the required guidance can instead be supplied by a cached planner output or a heuristic template. Thus, guidance is always available to RELOOP-I, while an additional online planner call is not always required. The final stopping decision remains conditioned on the retrieved evidence through the sufficiency signal.

Answer synthesis and optional refinement. Upon termination at step τ, the canonicalizer κ converts M_τ into a compact, provenance-preserving evidence package containing selected snippets and their source identifiers, levels, and alignment metadata. The head module then produces an answer grounded in the retrieved evidence:

$$\hat{y} = \mathcal{H}(q, \kappa(M_\tau); g)\,. \tag{5}$$

Although g provides task-level guidance, the head module is instructed to ground its answer in the canonicalized evidence $\kappa(M_\tau)$. An optional verifier ξ examines the evidence package for inconsistencies. If a contradiction is detected, RELOOP-I resumes for at most Δ additional steps with refined guidance g' and a remaining budget B'; otherwise, $\hat{y}$ is returned.

4 Instantiating RELOOP with Open-Source LLMs

This section describes the training instantiation of RELOOP-I with open-source language models. Each retrieval trajectory is conditioned on a guidance plan g, constructed according to the modes described in Section 3.4.

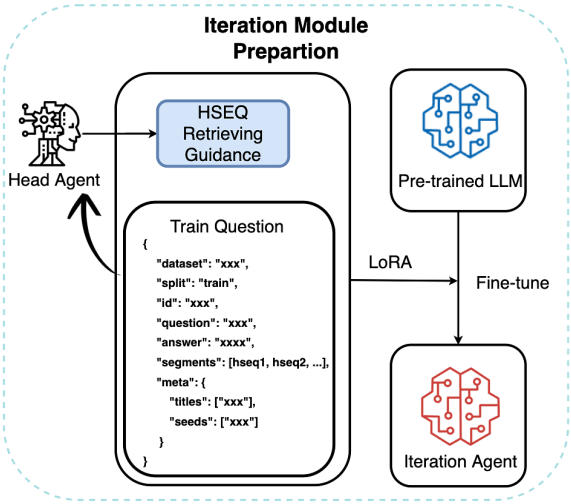


Figure 2: Fine-tuning RELOOP-I from heterogeneous QA instances. Each training trajectory is conditioned on guidance g and provides action and sufficiency supervision for LoRA adaptation.

4.1 Fine-tuning RELOOP-I

Training tuples and supervision. RELOOP-I is trained on tuples $(q, \mathrm{type}, S_h, g, A^\star)$, where type is an optional coarse question label, g is the guidance plan, and $A^\star = \{(a_t^\star, s_t^\star)\}_{t=1}^{\tau^\star}$ is a target retrieval trajectory. Each step contains an evidence-selection or expansion action $a_t^\star$ and a binary sufficiency target $s_t^\star$, with $\tau^\star = \min\{t : s_t^\star = 1\}$. When explicit trajectories are unavailable, weak-positive evidence $P^\star \subseteq S_h$ is induced through high-precision matching between gold answers or available oracle spans and segment content, optionally augmented by lexical overlap with q. A target trajectory is then synthesized by greedily selecting or expanding segments from $P^\star$ under the resource budget, and the terminal step is labelled as sufficient. Construction details are provided in App. A.2.

Policy learning. The iterator π_θ is adapted from a base open-source LLM using Low-Rank Adaptation (LoRA) (Hu et al., 2021) (App. A.3.4). Given $\text{state}_t = (q, S_h, M_t, C_t, g, B_t)$, training jointly optimizes action prediction and sufficiency-controlled stopping. With teacher forcing for $t < \tau^\star$, aiming:

$$\min_\theta \quad \mathbb{E}\left[\sum_{t=1}^{\tau^\star} \ell_{\text{act}}(\pi_\theta(\cdot \mid \text{state}_t), a_t^\star) + \lambda \ell_{\text{stop}}(\pi_\theta(\cdot \mid \text{state}_t), s_t^\star)\right]. \tag{6}$$

When $A^\star$ is synthesized from weak-positive evidence, confidence-based step weights reduce the contribution of uncertain supervision (App. A.2).

5 Experiment

RELOOP is evaluated on four benchmark datasets covering five QA tasks across text, table–text, and knowledge-graph evidence. Answer quality is measured using accuracy (Acc) and F1. Efficiency is evaluated using the average number of retrieval steps and online wall-clock latency; the accounting of guidance generation and offline HSEQ construction is specified in Section 5.7.

5.1 Experiment Setup

Table 3: Datasets used in our study (modality abbreviations: T=Text, Tb=Table, KG=Knowledge Graph).

Dataset	Modality	#Train	#Validation	#Test
HotpotQA	T	90447	7405	7405
TAT-QA	Tb + T	13,251	1644	1,663
HybridQA	Tb + T	62,682	3,466	3463
MetaQA-2Hop	KG	119,986	14,872	14,872
MetaQA-3Hop	KG	17,482	14,274	14,274

Benchmarks. RELOOP is evaluated on four benchmark datasets covering five QA tasks: *HotpotQA (Yang et al., 2018)* (multi-hop reasoning over Wikipedia text), *TAT-QA (Zhu et al., 2021)* (table-centric financial QA with accompanying paragraphs and numerical operations), *HybridQA (Chen et al., 2020)* (Wikipedia tables linked to passages requiring cross-format grounding), and *MetaQA (Zhang et al., 2018)* over a Wikidata-style KG. For MetaQA, the 2-hop and 3-hop tasks are used, since they require non-trivial relational evidence acquisition.

Baselines. The evaluation includes three classes of comparison systems. **LLM-only** baselines directly answer questions from raw source contexts using the corresponding backbone without iterative retrieval. **Table–text RAG** baselines include TAT-LLM (Zhu et al., 2024), TableRAG (Yu et al., 2025), ODYSSEY (Agarwal et al., 2025), TTQA-RS (Bardhan et al., 2024), and HippoRAG (Jimenez Gutierrez et al., 2024). **Text/KG multi-hop retrieval** baselines include Graph-constrained Reasoning (GcR) (Luo et al., 2024b), Think-on-Graph (ToG) (Ma et al., 2024), and AdaptiveRAG (Jeong et al., 2024). Because not all baselines support all evidence formats, each method is reported only on its applicable task setting.

Comparison protocol. Since not all baselines support all evidence formats, each method is reported only on its applicable tasks. Sources of baseline results, evaluation splits, backbones, and available inference settings are documented in App. A.4. Controlled ablations use matched RELOOP settings to isolate individual design components.

RELOOP configurations. RELOOP reports the best and median iterator–head configurations over the evaluated open-source model pairs. Complete backbone choices, valid pairs, hyperparameters, and inference budgets are provided in App. A.3. Unless otherwise stated, standard RELOOP inference includes guidance g; its construction mode is specified where accuracy or latency is reported.

5.2 Main Results

Table 4 reports answer quality across the five QA tasks. RELOOP achieves the highest accuracy on all tasks and the highest F1 on four of the five tasks. On HybridQA, RELOOP obtains the best accuracy while its F1 is marginally below HippoRAG (Jimenez Gutierrez et al., 2024). The median iterator–head configuration also remains competitive, outperforming the reported baselines on HotpotQA and both MetaQA tasks. These results indicate that the HSEQ-based retrieval procedure performs consistently across text, table–text, and KG evidence settings.

5.3 Component and Structure Ablations

Table 5 evaluates the contributions of RELOOP's training, guidance, representation, and navigation mechanisms. Standard RELOOP always conditions retrieval on guidance; *w/o guidance* is included only as a diagnostic setting. Removing SFT substantially degrades performance, showing that RELOOP-I benefits from explicit action and sufficiency supervision. Replacing planner-generated guidance with heuristic guidance produces a smaller degradation, indicating that useful

Table 4: Overall QA performance on heterogeneous benchmarks. Shaded cells (N/A) indicate the method is not applicable to that benchmark; gray dashes (–) indicate metric not reported. The best RELOOP result uses Qwen3-4B-Instruct-2507 as RELOOP-I and Falcon-H1-7B-Instruct as RELOOP-H. RELOOP reports both the best and median valid iterator–head configurations to distinguish upper-bound model selection from typical performance.

Method	HybridQA		TAT-QA		HotpotQA		MetaQA-2hop		MetaQA-3hop	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
LLM-only (direct QA)										
Falcon3-10B-instruct	22.4	–	35.2	–	16.5	–	43.0	–	39.8	–
Falcon-H1-7B-Instruct	32.9	–	43.7	–	21.1	–	48.3	–	44.6	–
Llama-3.1-8B-Instruct	28.1	–	37.6	–	14.6	–	37.8	–	31.9	–
Qwen3-4B-Instruct-2507	30.3	–	42.1	–	17.8	–	42.2	–	38.5	–
RAG-based methods (single-pass / agentic baselines)										
TAT-LLM (Table task)	N/A	N/A	73.1	81.0	N/A	N/A	N/A	N/A	N/A	N/A
TableRAG (Table task)	47.5	63.1	61.9	68.6	N/A	N/A	N/A	N/A	N/A	N/A
ODYSSEY (Text task)	51.5	66.0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
TTQA-RS (Text task)	62.3	70.6	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
HippoRAG (Table + text)	65.8	**72.4**	70.1	74.9	53.2	55.7	N/A	N/A	N/A	N/A
Graph-constrained Reasoning (GcR)	N/A	N/A	N/A	N/A	39.2	41.6	86.7	88.1	83.2	80.6
Think on Graph (ToG)	N/A	N/A	N/A	N/A	43.1	44.7	83.2	84.8	81.1	78.5
AdaptiveRAG (Graph task)	N/A	N/A	N/A	N/A	50.3	52.5	88.2	90.1	84.5	85.7
Our method: RELOOP										
RELOOP (best)	**66.4**	72.1	**75.7**	**83.5**	**56.3**	**58.6**	**95.9**	**91.1**	**93.4**	**88.3**
RELOOP (median)	63.9	70.8	73.2	79.6	55.4	57.1	93.2	89.7	90.1	86.6

Table 5: Component and structure ablations of RELOOP. The *w/o guidance* condition is a diagnostic variant; standard RELOOP always uses guidance. For representation/navigation ablations, iterator/head backbones, guidance settings, and inference budgets are matched to the full model.

Variant	**HybridQA**		**TAT-QA**		**HotpotQA**		**MetaQA-2hop**		**MetaQA-3hop**	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
RELOOP (full)	**66.4**	**72.1**	**75.7**	**83.5**	**56.3**	**58.6**	**95.9**	**91.1**	**93.4**	**88.3**
Training and guidance										
w/o SFT	57.3	65.7	60.4	66.9	46.5	47.8	78.3	80.1	74.6	72.5
w/o guidance	59.2	62.6	68.8	75.1	50.5	51.2	82.4	83.0	79.2	73.8
heuristic guidance	63.8	67.3	70.4	79.9	54.7	56.1	87.3	85.4	83.9	86.1
Representation and navigation										
Flat-SEQ + Same Iterator	60.3	64.1	66.8	76.2	53.1	54.9	80.4	80.9	77.6	78.4
HSEQ w/o Structural Expansion	62.8	66.9	69.5	78.6	53.9	55.8	85.1	83.2	81.4	82.7

retrieval guidance need not always require an online planner call. The representation and navigation variants directly examine the role of HSEQ. *Flat-SEQ + Same Iterator* removes HSEQ structural links while retaining the same iterative retrieval setting, whereas *HSEQ w/o Structural Expansion* preserves typed segments but disables neighborhood-expansion actions. The resulting performance differences quantify whether RELOOP's gains arise from structured representation, structure-respecting navigation, or their combination. The target-task exclusion experiment further separates policy transfer from task-specific supervision: omitting HybridQA trajectories lowers its accuracy/F1 from 66.4/72.1 to 62.8/68.2, while the no-SFT iterator obtains 65.7 F1. The result indicates both crosstask transfer and a measurable benefit from targettask trajectories.

Table 6: Reliability of sufficiency-controlled stopping. Premature Stop(Pre.) denotes the proportion of trajectories terminated before oracle sufficient evidence is covered. Extra Steps(+Step) measures the average number of steps beyond the oracle stopping step. Oracle Gap(Gap) reports the answer-quality difference between predicted and oracle stopping.

Task	**Prec.↑**	**Rec.↑**	**Pre. ↓**	**+Step ↓**	**Gap ↓**
HotpotQA	0.84	0.86	0.14	0.31	3.1
HybridQA	0.88	0.91	0.10	0.44	2.3
TAT-QA	0.79	0.82	0.18	0.52	4.2
MetaQA-2hop	0.94	0.96	0.05	0.17	1.1
MetaQA-3hop	0.90	0.93	0.08	0.29	1.8
Average	0.87	0.90	0.11	0.35	2.5

Table 7: Efficiency analysis. Panel A reports online inference under cached guidance; Panel B decomposes the cost of the best RELOOP pair on HotpotQA under different guidance construction modes.

Panel A: Online latency-sensitive inference

Method	HotpotQA		MetaQA-2hop		MetaQA-3hop	
	Steps	Lat. (ms)	Steps	Lat. (ms)	Steps	Lat. (ms)
LLM-only	1	3259 $\pm$490	1	2433 $\pm$384	1	3429 $\pm$545
Think-on-Graph	13.28	22826 $\pm$6810	11.73	15829 $\pm$4710	16.58	23901 $\pm$7292
RELOOP (latency-sensitive, best pair)	4.00	6247 $\pm$375	3.27	5528 $\pm$1376	4.11	10031 $\pm$2552
RELOOP (latency-sensitive, median pair)	4.17	11927 $\pm$2910	3.76	9201 $\pm$2275	4.59	12825 $\pm$3241

Panel B: RELOOP cost decomposition on HotpotQA

Guidance / preprocessing mode	Guidance cost	Retrieval + answer	Total / status
Heuristic guidance	2.0 ± 0.5 ms	6247 ± 375 ms	6249 ± 375 ms
Cached guidance	0.1 ± 0.0 ms	6247 ± 375 ms	6247 ± 375 ms
Cold-start planner	5247 ± 1586 ms	6247 ± 375 ms	11494 ± 1630 ms
Offline HSEQ build(one-off offline cost)	–	–	1.7 ± 0.1 s

5.4 Reliability of Sufficiency-Controlled Stopping

Because retrieval termination directly affects both answer quality and inference cost, Table 6 evaluates the reliability of the sufficiency signal. For each task, an oracle stopping step $\tau^\star$ is defined as the earliest step at which the selected evidence covers the available gold or derived sufficient evidence set. Dataset-specific construction rules for oracle evidence are provided in App. A.5. Stopping is most reliable on the KG tasks (MetaQA), where sufficient evidence corresponds to a clearly bounded relational neighborhood, and least reliable on TAT-QA, where the remaining errors are primarily associated with numerical-derivation questions whose sufficiency cannot be judged from surface evidence coverage alone.

5.5 Robustness to Incorrect Guidance

Table 8 evaluates guidance perturbations on one representative task per evidence format. Compared with planner guidance, corrupted or irrelevant guidance reduces F1 and increases retrieval steps. Nevertheless, RELOOP retrieves sufficient evidence in 86–95% of perturbed cases, indicating that inaccurate guidance can be corrected through evidence-conditioned iteration.

5.6 Accuracy–Efficiency Trade-off Across Model Pairs

Table 13 in App. A.8 reports the complete results across iterator–head configurations. On HybridQA, larger Qwen-based iterator configurations provide the strongest answer quality, whereas lightweight Llama/Falcon configurations reduce iteration depth and latency at the cost of lower accuracy. This trade-off indicates that RELOOP supports different operating points depending on whether answer quality or inference cost is prioritised.

Table 8: Robustness to incorrect guidance on representative tasks. Recovery denotes retrieval of sufficient evidence under perturbed guidance.

Task	Guidance setting	F1	Steps	Recovery
HotpotQA	Planner	58.6	4.00	–
	Heuristic	56.1	4.32	–
	Corrupted	53.4	4.63	0.89
	Irrelevant	54.1	4.55	0.91
HybridQA	Planner	72.1	3.71	–
	Heuristic	67.3	4.04	–
	Corrupted	67.2	4.33	0.92
	Irrelevant	68.6	4.10	0.95
MetaQA-3hop	Planner	88.3	4.59	–
	Heuristic	86.1	4.88	–
	Corrupted	83.0	5.21	0.86
	Irrelevant	83.4	5.06	0.88

5.7 Online Efficiency Analysis

Table 7 separates latency-sensitive online inference from cold-start planning and offline HSEQ construction. Under cached guidance, RELOOP requires substantially fewer retrieval steps than ToG, and the best iterator–head pair is more than twice as fast on all three reported tasks. Panel B further shows that heuristic and cached guidance introduce negligible query-time overhead, whereas cold-start planner generation incurs an additional online cost. HSEQ construction is performed once offline and is excluded from per-query latency. Cached guidance assumes repeated or offline-preprocessed queries; heuristic guidance is the live low-overhead alternative, whereas the cold-start row includes online planner generation.

6 Conclusion

This paper presents **RELOOP**, a framework for heterogeneous question answering that represents text, tables, and knowledge graphs through a reversible hierarchical sequence (**HSEQ**) and performs sufficiency-controlled iterative retrieval over this unified interface. HSEQ preserves provenance and structural links for evidence navigation, while RELOOP-I selects and expands compact evidence sets under resource budgets before RELOOP-H synthesizes the final answer from canonicalized evidence. Experiments across text, table–text, and KG benchmarks demonstrate strong answer quality and efficient multi-hop retrieval. Controlled ablations confirm the contributions of HSEQ structure, structural expansion, supervised iteration, and guidance; further analyses show reliable sufficiency-based stopping and substantial robustness to incorrect guidance. By combining a shared cross-format evidence interface with budget-aware retrieval, RELOOP provides an effective and auditable approach to heterogeneous QA.

Limitations

RELOOP is currently evaluated on static, single-turn QA benchmarks covering text, table–text, and knowledge-graph evidence. Although these settings provide diverse testbeds for heterogeneous retrieval and reasoning, real-world applications may involve continuously updated sources and multi-turn interactions. Extending HSEQ-based retrieval to such dynamic settings, together with broader source formats and deployment conditions, is an important direction for future work.

Ethical Considerations

RELOOP does not introduce new human-subject data collection, but its benchmark corpora may inherit factual errors, social biases, or references to identifiable entities from their original sources. HSEQ provenance improves traceability but does not guarantee factual correctness or eliminate hallucinations arising from incomplete retrieval or inaccurate guidance. Consequently, answers should be externally verified before use in high-stakes settings. Although controlled retrieval reduces unnecessary inference steps, the computational cost of training and evaluation remains a consideration for large-scale deployment.

Acknowledgments

This research was partially supported by the Technology Innovation Institute (TII), Abu Dhabi, United Arab Emirates, which conducts cutting-edge applied research in artificial intelligence and advanced technologies. Also, this research used the NVIDIA H100 cluster provided by SHARON AI.

References

Ankush Agarwal, Ganesh S, and Chaitanya Devaguptapu. 2025. Hybrid graphs for table-and-text based question answering using llms. *Preprint*, arXiv:2501.17767.

Jayetri Bardhan, Bushi Xiao, and Daisy Zhe Wang. 2024. Ttqa-rs-a break-down prompting approach for multi-hop table-text question answering with reasoning and summarization. *arXiv preprint arXiv:2406.14732*.

Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Wang. 2020. Hybridqa: A dataset of multi-hop question answering over tabular and textual data. *arXiv preprint arXiv:2004.07347*.

Xinyue Chen, Pengyu Gao, Jiangjiang Song, and Xiaoyang Tan. 2024. Hiqa: A hierarchical contextual augmentation rag for multi-documents qa. *arXiv preprint arXiv:2402.01767*.

Yiqun Chen, Lingyong Yan, Weiwei Sun, Xinyu Ma, Yi Zhang, Shuaiqiang Wang, Dawei Yin, Yiming Yang, and Jiaxin Mao. 2025. Improving retrieval-augmented generation through multi-agent reinforcement learning. *arXiv preprint arXiv:2501.15228*.

Philipp Christmann and Gerhard Weikum. 2024. Rag-based question answering over heterogeneous data and text. *arXiv preprint arXiv:2412.07420*.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. 2024. Trace the evidence: Constructing knowledge-grounded reasoning chains for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8472–8494.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Michael Glass, Gaetano Rossiello, Md Faisal Mahbub Chowdhury, Ankita Rajaram Naik, Pengshan Cai, and Alfio Gliozzo. 2022. Re2g: Retrieve, rerank, generate. *arXiv preprint arXiv:2207.06300*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2024. Grag: Graph retrieval-augmented generation. *arXiv preprint arXiv:2405.16506*.

Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025. Ket-rag: A cost-efficient multi-granular indexing framework for graph-rag. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 1003–1012.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403*.

Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992.

Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems*, 37:59532–59569.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Preprint*, arXiv:2005.11401.

Pei Liu, Xin Liu, Ruoyu Yao, Junming Liu, Siyuan Meng, Ding Wang, and Jun Ma. 2025. Hm-rag: Hierarchical multi-agent multimodal retrieval augmented generation. *arXiv preprint arXiv:2504.12330*.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. 2024a. Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, page 2039–2056. Association for Computational Linguistics.

Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. 2024b. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. *arXiv preprint arXiv:2410.13080*.

Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Dinh Phung, Chen Gong, and Shirui Pan. 2025. Gfm-rag: graph foundation model for retrieval augmented generation. *arXiv preprint arXiv:2502.01113*.

Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. *arXiv preprint arXiv:2407.10805*.

Costas Mavromatis and George Karypis. 2024. Gnn-rag: Graph neural retrieval for large language model reasoning. *arXiv preprint arXiv:2405.20139*.

Andrew Reynolds and Felix Corrigan. 2024. Improving real-time knowledge retrieval in large language models with a dns-style hierarchical query rag. *Authorea Preprints*.

Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. 2025. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint arXiv:2501.09136*.

Ruiyi Yang, Hao Xue, Imran Razzak, Hakim Hacid, and Flora D Salim. 2025. Beyond single pass, looping through time: Kg-irag with iterative knowledge retrieval. *arXiv preprint arXiv:2503.14234*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*.

Wenhao Yu. 2022. Retrieval-augmented generation across heterogeneous knowledge. In *Proceedings of the 2022 conference of the North American chapter of the association for computational linguistics: human language technologies: student research workshop*, pages 52–58.

Xiaohan Yu, Pu Jian, and Chong Chen. 2025. Tablerag: A retrieval augmented generation framework for heterogeneous document reasoning. *arXiv preprint arXiv:2506.10380*.

Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander Smola, and Le Song. 2018. Variational reasoning for question answering with knowledge graph. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. Tat-qa: A question answering benchmark on a hybrid of tabular and textual content in finance. *arXiv preprint arXiv:2105.07624*.

Fengbin Zhu, Ziyang Liu, Fuli Feng, Chao Wang, Moxin Li, and Tat Seng Chua. 2024. Tat-llm: A specialized language model for discrete reasoning over financial tabular and textual data. In *Proceedings of the 5th ACM International Conference on AI in Finance*, pages 310–318.

Yijie Zhu, Haojie Zhou, Wanting Hong, Tailin Liu, and Ning Wang. 2026. Reap: Enhancing rag with recursive evaluation and adaptive planning for multi-hop question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 35230–35238.

A Appendix

A.1 Theoretical Properties of RELOOP

A.1.1 Preliminaries and Assumptions

Segment schema. An HSEQ is a finite ordered sequence S_h of segments $s = (\mathrm{id}(s), \ell(s), p(s), c(s), \mu(s))$. Here ℓ is a level tag; p is a parent pointer with $p(s) = \perp$ if s is a root; c is content; μ is metadata, possibly including `offsets` (for text), `schema` and row indices (for tables), and triplet fields (for KGs).

Modality-specific metadata. A modality-aware adapter τ maps a heterogeneous corpus D to an HSEQ representation $S_h = \tau(D)$. For text, $\mu(s)$ records source identifiers and character offsets; for tables, it records table identifiers and row/column coordinates; for KGs, it records entity/relation identifiers and adjacent relational neighborhoods.

Source-level traceability. HSEQ uses the term *reversible* to denote evidence-level traceability rather than full-corpus reconstruction. For every selected segment s, $\mathrm{id}(s)$ and $\mu(s)$ retain the source identifiers and alignment metadata required to locate its originating evidence unit.

Seeded candidate windows. For each question, the initial candidate window is constructed as

$$C_0 = \mathrm{SeedWindow}(q, S_h, g; W, B_0, \rho),$$

where query and guidance signals identify candidate segments or regions before iteration begins. The ordering ρ determines the presentation order within seeded and structurally expanded candidates, rather than forcing a scan from the beginning of the full HSEQ sequence. Subsequent windows are obtained by refreshing unseen candidates and adding segments reached through the structure-aware neighborhood operators defined by HSEQ.

Source-level traceability. HSEQ uses the term *reversible* to denote evidence-level traceability rather than full-corpus reconstruction. For every selected segment s, $\mathrm{id}(s)$ and $\mu(s)$ retain the source identifiers and alignment metadata required to locate its originating evidence unit, such as text offsets, table coordinates, or KG entity/relation identifiers.

A.2 Weak-Positive Labeling and Sufficiency Supervision

Derived sufficient evidence. For each instance, a derived sufficient evidence set $E^{\star}_{\mathrm{der}} \subseteq S_h$ is constructed from available answer and supporting-evidence annotations. When explicit supporting evidence is unavailable, weak-positive segments are induced through high-precision matching between gold answers or oracle spans and segment content, optionally supplemented by question–segment lexical overlap.

Dataset-specific construction. For HotpotQA, $E^{\star}_{\mathrm{der}}$ is derived from annotated supporting facts. For HybridQA, it contains answer-linked table and passage evidence when available. For MetaQA, it corresponds to the supporting relational path leading to the answer entity. For TAT-QA, it is constructed from available table/text supervision and answer-linked segments; numerical-derivation questions may provide weaker supervision because the final answer need not appear verbatim in an individual segment.

Trajectory and stopping labels. Given $E^{\star}_{\mathrm{der}}$ and the per-step selection budget, a target trajectory is synthesized by selecting or expanding relevant segments until the derived sufficient evidence is covered. The target stopping label and oracle stopping step are

$$s^{\star}_t = \mathbb{I}[E^{\star}_{\mathrm{der}} \subseteq M_t]\,,$$
$$\tau^{\star} = \min\{t : E^{\star}_{\mathrm{der}} \subseteq M_t\}\,.$$

When supervision is weakly derived, confidence-based step weights reduce the loss contribution of uncertain selections.

Stopping metrics. Stop Precision measures the proportion of predicted stops for which $E^{\star}_{\mathrm{der}}$ has already been covered; Stop Recall measures the proportion of instances in which RELOOP-I stops after such evidence becomes available. Premature Stop denotes termination before $\tau^{\star}$, Extra Steps measures $\max(0, \hat{\tau} - \tau^{\star})$, and Oracle Gap reports

the answer-quality difference between predicted and oracle stopping.

A.2.1 Canonicalization and Evidence Traceability

Definition 1 (Canonicalizer)**.** *A canonicalizer κ maps a selected segment set $M \subseteq S_h$ to an evidence package*

$$\kappa(M) = \left\{ \left(\mathrm{id}(s), \ell(s), \bar{c}(s), \bar{\mu}(s) \right) : s \in M \right\},$$

where $\bar{c}(s)$ is a readable content view of $c(s)$ and $\bar{\mu}(s)$ retains the source-alignment metadata required to locate the originating evidence unit, such as text offsets, table coordinates, or KG entity/relation identifiers. The canonicalizer is traceability preserving *if each tuple in $\kappa(M)$ retains the unique segment identifier and source locator associated with its originating segment.*

Proposition 1 (Evidence traceability under canonicalization)**.** *If HSEQ assigns each segment $s \in S_h$ a unique identifier and source locator, and κ is traceability preserving, then every evidence item in $\kappa(M)$ can be mapped uniquely to a selected segment in M and traced to its originating evidence unit in the underlying source.*

Proof. Let

$$z = \left(\mathrm{id}(s), \ell(s), \bar{c}(s), \bar{\mu}(s) \right) \in \kappa(M)$$

be any canonicalized evidence item. By construction of κ, z retains the identifier $\mathrm{id}(s)$ of its originating segment. Since segment identifiers are unique in S_h, there exists exactly one segment $s \in M$ associated with z. Furthermore, traceability preservation ensures that $\bar{\mu}(s)$ retains the source locator required by the corresponding modality-specific adapter: offsets for text, coordinates for tables, or entity/relation identifiers for KGs. Therefore, z can be traced uniquely first to its selected HSEQ segment and then to its originating source-level evidence unit. □

A.2.2 Seeded Candidate Construction

Given a question q, guidance g, and HSEQ S_h, the initial candidate window is constructed as

$$C_0 = \mathrm{SeedWindow}(q, S_h, g; W, B_0, \rho).$$

Unlike a sequential scan over the full HSEQ representation, SeedWindow initializes retrieval from query- and guidance-relevant segments. Subsequent candidate sets are refreshed from unseen segments within the seeded regions and from structural neighborhoods reached through HSEQ expansion operators. The bounded window size limits the context presented to RELOOP-I at each step, while the empirical efficiency and stopping behaviour are evaluated in Tables 7 and 6.

A.2.3 Scope of the Formal Specification

The formal specification characterizes two design properties of RELOOP. First, HSEQ and the canonicalizer preserve source-level traceability: selected evidence items retain the identifiers and alignment metadata required to locate their originating text spans, table entries, or KG relations. Second, RELOOP-I operates over query- and guidance-seeded candidate regions together with HSEQ-induced structural neighborhoods, rather than scanning the full sequence from its beginning. These properties define the interface exposed to retrieval and answer synthesis, but do not constitute a guarantee of answer correctness or complete evidence recovery. The practical reliability of structured navigation, sufficiency-controlled stopping, and recovery from incorrect guidance is therefore evaluated empirically in Tables 5, 6, and 8.

A.3 Implementation Details

A.3.1 Agent models used for RELOOP

Models used for both iteration agent and head agent are shown in Table 9, grouped by size. Most experiments are done by using small and medium models (as of the result shown in main text).

A.3.2 Iteration-Agent Prompts and Output Schema

System instruction. The iteration agent is conditioned with a concise, role-defining system message:

```
You are an iteration agent
working over a hierarchical
sequence (H-Seq).
Given a question and a list of
candidate segments (each with an
id and text)
select the top-k segment_ids
that best support answering the
question.
Then decide if the selected
evidence is sufficient to stop.
Return ONLY compact JSON with
keys:  type, args.segment_ids,
args.strategy, args.top_k,
sufficiency.
WITHOUT ANY EXPLAINATION.
```

Prompt template. Each training step uses a structured multi-section prompt:

Table 9: Iteration-agent and head agent base models grouped by size.

Group	Model (HF id)
SMALL	tiiuae/Falcon-H1-0.5B-Instruct tiiuae/Falcon-H1-1.5B-Instruct tiiuae/Falcon3-1B-instruct meta-llama/Llama-3.2-1B-Instruct deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B
MEDIUM	tiiuae/Falcon3-3B-instruct tiiuae/Falcon-H1-3B-Instruct Qwen/Qwen3-4B-Instruct-2507 tiiuae/Falcon3-7B-instruct tiiuae/Falcon-H1-7B-Instruct meta-llama/Llama-3.2-3B-Instruct meta-llama/Meta-Llama-3-8B-Instruct meta-llama/Llama-3.1-8B-Instruct deepseek-ai/DeepSeek-R1-Distill-Qwen-7B deepseek-ai/DeepSeek-R1-Distill-Llama-8B
LARGE	tiiuae/Falcon3-10B-instruct tiiuae/Falcon-H1-34B-Instruct Qwen/Qwen3-30B-A3B-Instruct-2507 deepseek-ai/DeepSeek-R1-Distill-Qwen-14B deepseek-ai/DeepSeek-R1-Distill-Qwen-32B deepseek-ai/DeepSeek-R1-Distill-Llama-70B meta-llama/Llama-3.1-70B-Instruct

```
### Instruction
{system-instruction}
### Question
{q}
### Guidance
{g(q,type)}
### Selected-So-Far
- [seg_id] truncated_content
...
### Candidate-Window
- [seg_id] truncated_content
...
### Output (JSON)
```

Only identifiers, levels, truncated content, and key metadata of segments are serialized.

Output schema. The agent must emit deterministic, machine-checkable JSON:

```
{ "type": "select", "args":
{ "segment_ids": [...],
"strategy": "guided_topk",
"top_k": k }, "sufficiency":
true/false }
```

No free-form text is allowed. This constraint simplifies supervision and evaluation.

Masking for SFT. During supervised fine-tuning, the loss is applied only to the *output* portion of the sequence (prompt tokens are masked), yielding a standard next-token objective over the action string while keeping inputs loss-free.

A.3.3 Guidance Generation and Caching

Head-generated guidance. A lightweight planner ("head") converts (q, type) into a short plan g that specifies: (i) what to retrieve first, (ii) optional branches, and (iii) a sufficiency hint. The planner is prompted with:

```
You are a planning assistant.
Given a question, write a
short retrieval plan for an
iteration agent selecting
evidence snippets. Specify
ONLY what to retrieve first,
possible branches, and when to
stop (sufficiency condition).
```

A short completion is generated and, if too brief or incomplete, a single continuation is requested to end with an explicit stop condition.

Heuristic templates. When a head is unavailable or for ablations, templates keyed by coarse patterns produce g, start with:

"Plan: retrieve a minimal set of highly relevant snippets; prefer concise facts."

Then add the following according to $\mathbb{Q}_{type}$:

- **Numeric**: Look for numeric mentions and table rows; stop when final number is explicit or corroborated.
- **Factoid (who/which/where/when)**: Focus on short spans that directly contain the answer; stop on a clear statement..
- **Binary**: Retrieve one-two definitive statements; stop when evidence strongly supports yes/no..

- **Default**: Prefer snippets naming key entities/relations; stop when answer is explicitly stated.

Caching. Guidance strings are cached per example using a stable key (dataset name and a hash of q) under a directory organized by head model id. Cache is consulted before running the head planner to reduce overhead.

Settings. The planning head is run with short outputs and deterministic decoding. A minimal-length heuristic is applied to avoid truncated guidance.

A.3.4 LoRA Adaptation and Optimization

Parameterization. The iteration agent is obtained by adding low-rank adapters to a base causal LLM. Adapters are attached to attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and MLP projections (`gate_proj`, `up_proj`, `down_proj`); vocabulary and positional embeddings are unchanged.

Default configuration. LoRA rank $r = 16$, scaling $\alpha = 32$, dropout 0.05, no bias; the language head is preserved as a save-module. Mixed-precision and 4-bit weight quantization (NF4 with double quantization) are used to reduce memory. Gradient checkpointing is enabled.

Training schedule. A cosine learning-rate schedule with warmup ratio 0.03 is used; batches are accumulated over several steps to match the target global batch size. Maximum input length is capped to a few thousand tokens; candidate windows and per-step k are tuned to respect the overall budget.

Mixture and curriculum. Examples are sampled across datasets by normalized weights; quotas are computed for a target mixed size and shuffled. A short-to-long curriculum increases the maximum number of steps T as training progresses.

Finetuning Parameters Table 10 shows the SFT hyperparameters for each model-size group.

A.3.5 Canonicalization and Sufficiency

Canonical evidence package. At termination, a modality-agnostic canonicalizer κ converts M_τ into a compact record set: $\kappa(M_\tau) = \{r_s\}_{s \in M_\tau}$, $r_s = (\texttt{id}, \texttt{lvl}, \texttt{uri}, \texttt{off}, \texttt{typ}, \texttt{snip}, \texttt{meta})$.

where `lvl=level`, `off=offsets`, `typ=source_type`, and `snip=snippet`. with the following contract: (i) **id**: globally unique, deterministically derived (*e.g.*, $\mathrm{sha1}(\texttt{uri}, \texttt{offsets})$); (ii) **uri**: source identifier with version (*e.g.*, document path or graph name); (iii) **offsets**: zero-based half-open character indices $[a, b)$ into the *original* source; for tables, $[i, j]$ denotes row/column coordinates; for KGs, `offsets`$= (-1, -1)$; (iv) **snippet**: a human-readable content aligned to sentence/field boundaries when possible; (v) **meta**: integrity and alignment helpers (`schema`, `time`, `source_version`, `sha1`). Duplicates are removed by $(\texttt{uri}, \texttt{offsets})$ and the package is *deterministically* ordered by `uri` then `offsets`. Typed views are derived on demand: *text* $\Rightarrow$ spans with section/paragraph ids; *table* $\Rightarrow$ `row_id`, `col_ids`, `schema`, `cell_coords`; *KG* $\Rightarrow$ (h, r, t) plus optional validity time.

Stopping signal. The sufficiency head outputs $s_t \in \{0, 1\}$ at each step. **Training targets** follow a coverage-based heuristic: $s_t^\star = 1$ if and only if the current M_t satisfies task-specific adequacy (*e.g.*, contains at least one gold-positive segment; achieves full slot coverage for table QA; or yields a unique answer span/number under a fixed head). For weak supervision, per-step weights downweight low-confidence positives (App. A.2). **Inference** uses a calibrated threshold τ on the model's sufficiency score $\hat{p}_t$ and enforces a minimum step count $T_{\min}$: stop at $\tau = \min\{t \geq T_{\min} : \hat{p}_t \geq \tau\}$ or when budget B is exhausted. Optionally, a lightweight contradiction checker triggers a one-shot refinement loop of at most Δ additional steps with tightened guidance g' and reduced budget B'. Thresholds $(\tau, T_{\min})$ are selected on the development split and may be calibrated via temperature scaling.

A.3.6 Reproducibility Notes

- **Seed and sampling.** A fixed seed is used for example subsampling and order shuffling.
- **Segment capping.** The number of serialized candidate segments per step is capped to respect the overall token budget; truncation is applied to `content` strings for display.
- **Budget control.** Global limits on steps, tokens, and optional tool calls are enforced; guidance encourages early sufficiency.
- **Hardware.** Experiments are run on maximum 4 `NVIDIA H200 Tensor Core GPU`. Mixed-precision and 4-bit quantization

Table 10: Supervised fine-tuning (SFT) hyperparameters for each model-size group. These settings apply to all models within the corresponding group.

Group	Target steps	Batch	GA	LR	ML	MS	Top-k	Mi	BF16
SMALL	12000	2	8	2.0×10^{-5}	3072	48	2	4	Yes
MEDIUM	9000	2	8	1.5×10^{-5}	3072	48	4	4	Yes
LARGE	4500	1	16	1.0×10^{-5}	2048	32	5	4	Yes

Notes. *Batch* is `-per_device_train_batch_size`. *GradAcc* is `-grad_accum`. *LR* is `-lr`. *ML*, *MS*, *Top-k*, *Mi* map to `-max_length`, `-max_segments`, `-top_k`, `-max_iters`. BF16 indicates `-bf16` enabled.

substantially reduce memory; typical training runs fit on a single GPU.

A.4 Baseline Details

For baselines we report results under their officially recommended settings. Table–text baselines (TAT-LLM (Zhu et al., 2024), TableRAG (Yu et al., 2025), ODYSSEY (Agarwal et al., 2025), TTQA-RS (Bardhan et al., 2024), and HippoRAG (Jimenez Gutierrez et al., 2024)) are evaluated on their supported table–text tasks (HybridQA, TAT-QA); text/KG multi-hop baselines (GcR (Luo et al., 2024b), ToG (Ma et al., 2024), AdaptiveRAG (Jeong et al., 2024)) are evaluated on HotpotQA and MetaQA. Numbers are taken from the original papers where the evaluation split, metric (Acc/EM/F1), and backbone match our protocol; where a method does not report a metric on a task we mark it "–", and where a method is not applicable to an evidence format we mark it N/A. Because the cited systems use heterogeneous backbones (ranging from fine-tuned task-specific models to proprietary LLMs), the comparison is across reported operating points rather than a single shared backbone; the *LLM-only* block in Table 4 provides the same-backbone reference, since each LLM-only row uses the identical model that serves as the corresponding RELOOP-H head.

A.5 Oracle Sufficiency Labels

The oracle stopping step $\tau^\star$ in Table 6 is the earliest iteration at which the selected segment set M_t covers a *sufficient* evidence set, defined per modality:

- **Text (HotpotQA).** The gold supporting paragraphs annotated in the dataset; M_t covers the evidence when all gold supporting sentences are contained in selected segments (matched by document id and sentence offset).
- **Table–text (HybridQA, TAT-QA).** The gold answer cell/row plus any linked passage required by the annotation; coverage requires the gold row (matched by row index) and, for TAT-QA derivation questions, all operand cells named in the gold program.
- **KG (MetaQA).** The gold relation path from the topic entity to the answer entity; coverage requires every triplet on a shortest gold path to be present among selected triplet segments.

Table 11: Latency decomposition. Offline HSEQ construction is a one-time cost amortised across all queries and is excluded from the online latency in Table 7; cold-start planner generation is an optional per-query cost incurred only on cache misses.

Component	Regime	Cost
HSEQ construction (text)	offline, one-off	≈37k seg/s
HSEQ construction (KG)	offline, one-off	≈53k seg/s
Guidance (cached/heuristic)	online, per query	≈ 0.1 s
Guidance (cold-start planner)	online, per query	5.2–6.6 s
Iterator selection (per step)	online, per query	1.5–2.5 s
Head answering	online, per query	1.5–3.0 s

Stopping precision/recall are computed by treating the predicted stop event $(s_t{=}1)$ as a detector of "M_t first becomes sufficient at step t". Premature Stop is the fraction of trajectories that stop before $\tau^\star$; Extra Steps is $\mathbb{E}[\max(0, \tau - \tau^\star)]$; and Oracle Gap is the answer-quality difference (F1) between predicted-stop and oracle-stop evidence packages.

A.6 Latency Decomposition

Table 11 decomposes per-query cost into the components that the main-text efficiency table (Table 7) excludes from online latency: the one-off *offline* HSEQ construction (amortised over all queries against a corpus) and the optional *cold-start* planner call. HSEQ construction throughput is measured on a single CPU core; the online guidance row reports the per-query cost of the cached/heuristic path used in Table 7.

A.7 Notations

Table 12 lists all symbols used in main context.

A.8 Overall performance of RELOOP agent pairs

Table 13 shows the overall performance of RELOOP agent pairs on Hybrid-QA, containing Accuracy/F1 and Efficiency.

A.9 Example Using RELOOP

A.9.1 Concrete fragment examples.

Below we instantiate (ℓ, p, c, μ) for three modalities (all values are illustrative):

$s_{\text{text}} = (\text{id} = s_1,\ \ell = \texttt{sentence},\ p = p_1,\ c = \text{“...capital is Paris...”},$
$\mu = \{\texttt{source_id} = \texttt{doc_12},\ \texttt{offsets} = (128, 172),\ \texttt{time} = 1992\});$
$s_{\text{row}} = (\text{id} = r_3,\ \ell = \texttt{table_row},\ p = t_1,\ c = \text{“France | 67.4M | EU”},$
$\mu = \{\texttt{source_id} = \texttt{tbl_7},\ \texttt{schema} = \text{[country,pop,bloc]},\ \texttt{row} = 3\});$
$s_{\text{cell}} = (\text{id} = u_{3,2},\ \ell = \texttt{table_cell},\ p = r_3,\ c = \text{“67.4M”},$
$\mu = \{\texttt{source_id} = \texttt{tbl_7},\ \texttt{row} = 3,\ \texttt{col} = 2\});$
$s_{\text{kg}} = (\text{id} = k_9,\ \ell = \texttt{triplet},\ p = g_1,\ c = \text{“(Paris, capital_of, France)”},$
$\mu = \{\texttt{source_id} = \texttt{kg_2}\}).$

Segments are concatenated in parent-before-child order. This minimal contract enables structure-aware neighborhoods and budget-aware iteration without inspecting raw files.

A.9.2 Case Study: Guided Iterative Retrieval on HYBRIDQA

Setup. Query q: *“Who is the author of the novel that inspired the 2004 Russian film directed by Timur Bekmambetov?”* RELOOP-I (iterator): `Qwen3-4B-Instruct-2507`; RELOOP-H (head): `Falcon-H1-7B-Instruct`. Guidance mode: `head`; source: cache (latency $\approx$0.12 ms).

Head-generated guidance. The head planner emits a short plan: (i) identify the 2004 Russian film directed by Bekmambetov; (ii) locate the novel that inspired it; (iii) stop once the *author of that novel* is found. This plan is injected as a prefix and acts as a soft prior on where the iterator should probe first.

Guided iteration over S_h. The iterator consumes the guidance and operates over the HSEQ stream with a fixed window and top-k selection. Table 14 summarizes the six steps (all sufficiency flags were `false`; the loop terminates by budget).

Answer synthesis. After τ=6 iterations, the canonicalizer κ compacts the selected set M_τ (paragraph + corroborating table rows) into a provenance-preserving package (segment ids, levels, offsets, snippets). The head $\mathcal{H}$ is prompted *only* with $(q, \kappa(M_\tau))$ and outputs:

$$\hat{y} = \text{Sergei Lukyanenko.}$$

The prediction matches the gold answer (EM/F1 = 1.0). Runtime profile: selection latency $\approx$32,185 ms, head latency $\approx$1,826 ms, total $\approx$34,011 ms; number of iterations = 6.

Takeaway. Guidance steers the iterator to a high-yield paragraph in the first step, which already contains the sufficient evidence (film identity and source novel). Subsequent steps provide corroboration from structured rows. The provenance in $\kappa(M_\tau)$ makes the final answer auditable: the paragraph `p_6df9c849` explicitly ties *Night Watch* (2004, Bekmambetov) to the novel *Night Watch* by Sergei Lukyanenko, enabling concise and well-grounded answer synthesis by the head.

A.9.3 Case Study: Guided Iterative Retrieval on HOTPOTQA

Setup. Query q: *“Which style is the building located on the East Side of Midtown Manhattan that Robert Von Ancken appraised?”* RELOOP-I (iterator): `Qwen3-4B-Instruct-2507`; RELOOP-H (head): `Falcon-H1-7B-Instruct`. Guidance mode: `head`; source: generated online (latency $\approx$8,496 ms).

Head-generated guidance. The head planner issues a short plan: (i) identify buildings on the East Side of Midtown Manhattan connected to appraiser *Robert Von Ancken*; (ii) once the specific building is found, retrieve its architectural style; (iii) stop when the style is clearly linked to the appraised building.

Guided iteration over S_h. The iterator follows the guidance with a fixed window and top-k selection. Table 15 lists the six steps (all sufficiency flags `false`; termination by budget). Note that Step 1 already surfaces the key paragraph about the Chrysler Building.

Answer synthesis. After τ=5 iterations, the canonicalizer κ compacts the selected set M_τ (including `p_a73a8d8f` and the Von Ancken paragraph `p_658d6333`) into a provenance-preserving package. The head answers using only $(q, \kappa(M_\tau))$:

$$\hat{y} = \text{Art Deco-style skyscraper.}$$

The prediction matches the gold answer. Runtime profile: selection latency $\approx$32,153 ms, head latency $\approx$838 ms, total $\approx$41,487 ms; iterations = 5.

Takeaway. The head's guidance steers the iterator directly to a paragraph that states both the location (East Side of Midtown) and the architectural style (Art Deco) of the relevant building (Chrysler Building), while additional picks provide neighborhood and appraiser context. Provenance in $\kappa(M_\tau)$ supports auditable linking from the final answer to its evidence.

A.10 Statement for The Use of Large Language Models (LLMs)

We used large language models (LLMs) as general-purpose tools for *writing assistance* and *engineering support*. No text or code generated by an LLM was used verbatim without author review; we take full responsibility for the content.

Table 12: Notation used throughout the paper.

Symbol	Meaning
q	Natural-language query (question).
$D = \{(x_j, m_j)\}_{j=1}^{N}$	Heterogeneous corpus with items x_j and modality tags $m_j \in \{\texttt{text}, \texttt{table}, \texttt{kg}\}$.
m_j	Modality label for the j-th item (text / table / KG).
$\tau,\ \tau_m$	Modality-aware adapter; $\tau(D)$ produces the unified hierarchical sequence. τ_m is the adapter for modality m.
S_h	The **HSEQ** (hierarchical sequence): $S_h = \bigsqcup_j \tau_{m_j}(x_j) \in \mathcal{S}^*$.
$\mathcal{S}$	Segment universe. Each segment $s \in \mathcal{S}$ is a lightweight record.
$s = (\mathrm{id}(s), \ell(s), p(s), c(s), \mu(s))$	Segment fields: unique identifier, level tag (granularity), parent pointer, compact human-readable content, standardized metadata.
$\ell(s)$	Level tag (e.g., `document`/`paragraph`/`sentence`, `table_row`/`table_cell`, `triplet`/`subgraph`).
$p(s)$	Parent pointer (container linkage) encoding locality in the hierarchy.
$c(s)$	Compact content snippet (text span / serialized table row / triple).
$\mu(s)$	Metadata with fixed keys (e.g., `source_id`, `uri`, `offsets`/coordinates, `schema`, `time`).
π_θ	**RELOOP-I** iteration policy (LLM-based) with parameters θ; operates over (q, S_h) to select evidence iteratively.
$g = g(q, \mathrm{type})$	Short *guidance* prior (from planner/head or heuristics) shaping early exploration and stop notion.
$B,\ B_t$	Budget (global / per-step): token, tool-call, step, and/or latency limits.
M_t	Selected-evidence set at step t; $M^\star$ is the final selected set at termination.
C_t	Candidate window at step t (bounded by window size and ordering).
$k,\ W$	Top-k selection cap per step; window size W for the exposed candidate stream.
$T_{\max},\ T_{\min}$	Maximal and minimal number of iteration steps (cap and anti–early-stop).
ρ	Deterministic ordering over S_h levels (e.g., paragraph $\prec$ row $\prec$ sentence $\prec$ triplet) to form the stream.
$\mathcal{N}(\cdot)$	Structure-aware neighborhood operators (parent/child, row/column, KG relation hops).
$a_t,\ s_t$	Action at step t (e.g., select up to k segments and/or expand neighborhoods) and sufficiency prediction $s_t \in \{0, 1\}$.
Φ	Budget-aware sufficiency criterion queried by the iterator to trigger termination.
κ	Canonicalizer mapping M_τ to provenance-preserving evidence package (ids, levels, offsets/coordinates, snippets).
$\mathcal{H}$	**RELOOP-H** head module for answer synthesis from $(q, \kappa(M_\tau))$; can also generate guidance g.
ξ	Optional verifier; on contradiction detection, triggers a brief refinement loop with tightened g' and reduced B'.
$y,\ \hat{y}$	Gold answer and system prediction, respectively.
$E^\star$	Minimally sufficient evidence set (w.r.t. a fixed answerer) for q in D.
$\mathrm{Window}(\cdot),\ \mathrm{Refresh}(\cdot)$	Operators to expose a bounded candidate window and to advance it while removing already selected segments.
Δ	Max number of additional refinement steps if the verifier ξ requests a retry.

Table 13: Overall performance of RELOOP agent pairs on Hybrid-QA: Accuracy/F1 and Efficiency.

		Accuracy & F1		**Efficiency**	
Iteration Agent (RELOOP-I)	Head Agent (RELOOP-H)	Avg. Acc	Avg. F1	Steps ↓	Latency (ms) ↓
Llama-3.2-3B-Instruct	Falcon3-10B-instruct	60.4	62.8	2.08	12055.5
Qwen3-4B-Instruct-2507	Falcon3-10B-instruct	63.9	64.5	4.1	20577.5
Falcon3-3B-instruct	Falcon3-10B-instruct	59.3	61.1	2.6	10530.1
Llama-3.2-3B-Instruct	Llama-3.1-8B-Instruct	55.4	57.9	**2.11**	**8346.3**
Qwen3-4B-Instruct-2507	Llama-3.1-8B-Instruct	65.5	71.2	3.29	16503.2
Falcon3-3B-instruct	Llama-3.1-8B-Instruct	61.2	65.1	2.46	11616.7
Llama-3.2-3B-Instruct	Falcon-H1-7B-Instruct	58.7	63.9	2.41	12080.0
Qwen3-4B-Instruct-2507	Falcon-H1-7B-Instruct	**66.2**	**71.4**	3.71	21479.2
Falcon3-3B-instruct	Falcon-H1-7B-Instruct	56.1	58.6	2.25	11714.4
Llama-3.2-3B-Instruct	DeepSeek-R1-Distill-Qwen-7B	62.5	60.2	2.75	15073.7
Qwen3-4B-Instruct-2507	DeepSeek-R1-Distill-Qwen-7B	62.8	66.7	4.07	21094.8
Falcon3-3B-instruct	DeepSeek-R1-Distill-Qwen-7B	61.4	62.0	3.01	13709.7

Table 14: Stepwise selection (abridged). Segment ids prefixed by level: `p_` (paragraph), `row_` (table row).

Step	**Key picks (content excerpt)**	**Sufficient?**
1	`p_6df9c849`: "*Night Watch* (. . .) is a 2004 Russian . . . directed by Timur Bekmambetov. It is loosely based on the novel *The Night Watch* by Serg[ei Lukyanenko]. . . "	No
2	`p_c15173df`, `p_3bc4a108`, `p_54f6ef94`: contextual paragraphs ("List of Russian films of 2004", "2004" entries)	No
3	`row_a44a4a17`: table row confirming *Night Watch* with director "Timur Bekmambetov"	No
4–6	additional table rows from the same list (*Arie*, *Countdown*, *Dad or Papa*, etc.) providing film set context	Yes

Table 15: Stepwise selection (abridged). Segment ids prefixed by level: `p_` (paragraph).

Step	**Key picks (content excerpt)**	**Sufficient?**
1	`p_a73a8d8f`: "The *Chrysler Building* is an **Art Deco-style** skyscraper located on the East Side of Midtown Manhattan . . . "	No
2	`p_c01522d2`: "23 Beekman Place . . . apartment building . . . East Side of Midtown Manhattan . . . "	No
3	`p_7c2aa386`: "The Helmsley Building . . . Midtown Manhattan . . . "	No
4	`p_658d6333`: "*Robert Von Ancken* is a prominent New York City real estate appraiser . . . "	No
5	`p_e97ef7e6`: "Lenox Hill Neighborhood House . . . East Side of Manhattan . . . "	Yes